

Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе №2

Вычислительная математика

Вариант 8

Выполнил: Караганов Павел

Группа: Р3210

Проверил: Рыбаков Степан Дмитриевич

г. Санкт-Петербург,

2026

Цель работы

Целью данной лабораторной работы является изучение и программная реализация численных методов решения нелинейных уравнений и систем нелинейных уравнений. В рамках работы необходимо:

1 Вычислительная реализация задачи

1.1 Решение нелинейного уравнения

$$f(x) = 3x^3 + 1.7x^2 - 15.42x + 6.89 = 0.$$

$$\varepsilon = 10^{-2}$$

1.1.1 Отделение корней

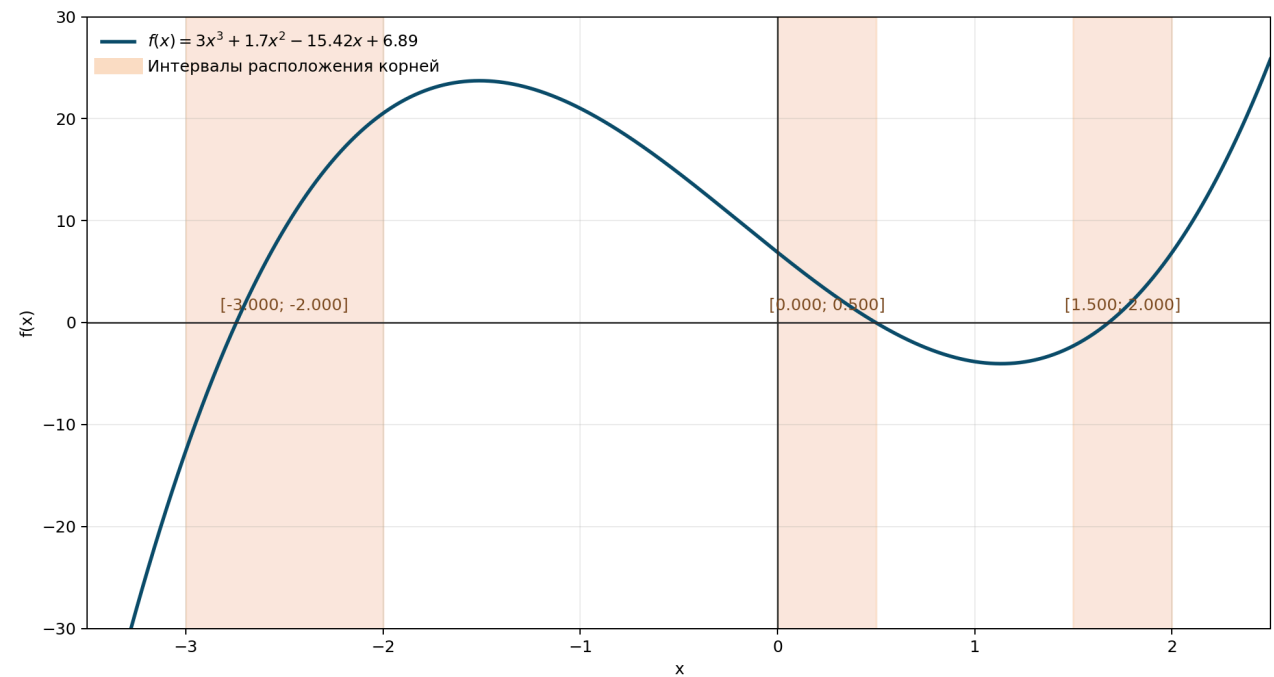
Для отделения корней вычислим значения функции в контрольных точках:

x	-3.000	-2.000	0.000	0.500	2.000
f(x)	-12.550	20.530	6.890	-0.020	6.850

Дополнительно: $f(1.500) = -2.290$.

Из смены знака получаем интервалы изоляции трех корней:

- $[-3.000; -2.000]$,
- $[0.000; 0.500]$,
- $[1.500; 2.000]$.



1.1.2 Уточнение левого корня методом хорд

Формула метода:

$$x_k = \frac{af(b) - bf(a)}{f(b) - f(a)}.$$

№ шага	a	b	x	f(a)	f(b)	f(x)	$ x_{k+1} - x_k $
1	-3.000	-2.000	-2.621	-12.550	20.530	4.983	-
2	-3.000	-2.621	-2.728	-12.550	4.983	0.684	0.108
3	-3.000	-2.728	-2.742	-12.550	0.684	0.086	0.014
4	-3.000	-2.742	-2.744	-12.550	0.086	0.011	0.002

Итог: $x \approx -2.744$, критерий точности выполнен, так как $|x_4 - x_3| = 0.002 < 0.01$.

1.1.3 Уточнение центрального корня методом секущих

Формула метода:

$$x_{k+1} = x_k - \frac{f(x_k)(x_k - x_{k-1})}{f(x_k) - f(x_{k-1})}.$$

№ итерации	x_{k-1}	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	0.000	0.500	0.499	-0.003	0.001

Итог: $x \approx 0.499$, критерий точности выполнен, так как $|x_{k+1} - x_k| = 0.001 < 0.01$.

1.1.4 Уточнение правого корня методом простой итерации

Выберем итерационную функцию:

$$\varphi(x) = x + \lambda f(x), \quad \lambda = -\frac{1}{\max_{x \in [1.5; 2]} |f'(x)|} = 0.03.$$

NB: Так как $f'([1.5, 2]) > 0$, то $\lambda = -\frac{1}{\max_{x \in [1.5; 2]} |f'(x)|}$

Тогда:

$$x_{k+1} = \varphi(x_k) = x_k - 0.03f(x_k).$$

Проверка условия сходимости на интервале $[1.5; 2.0]$:

$$f'(x) = 9x^2 + 3.4x - 15.42,$$

$$\varphi'(x) = 1 - 0.03f'(x).$$

На концах интервала:

$$\varphi'(1.5) \approx 0.702,$$

$$\varphi'(2.0) \approx 0.179.$$

Следовательно, $|\varphi'(x)| < 1$ на выбранном интервале, метод сходится.

№ итерации	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	2.000	1.795	2.029	0.206
2	1.795	1.734	0.898	0.061
3	1.734	1.707	0.438	0.027
4	1.707	1.694	0.223	0.013
5	1.694	1.687	0.116	0.007

Итог: $x \approx 1.687$, критерий точности выполнен, так как $|x_5 - x_4| = 0.007 < 0.01$.

1.1.5 Результат первой части

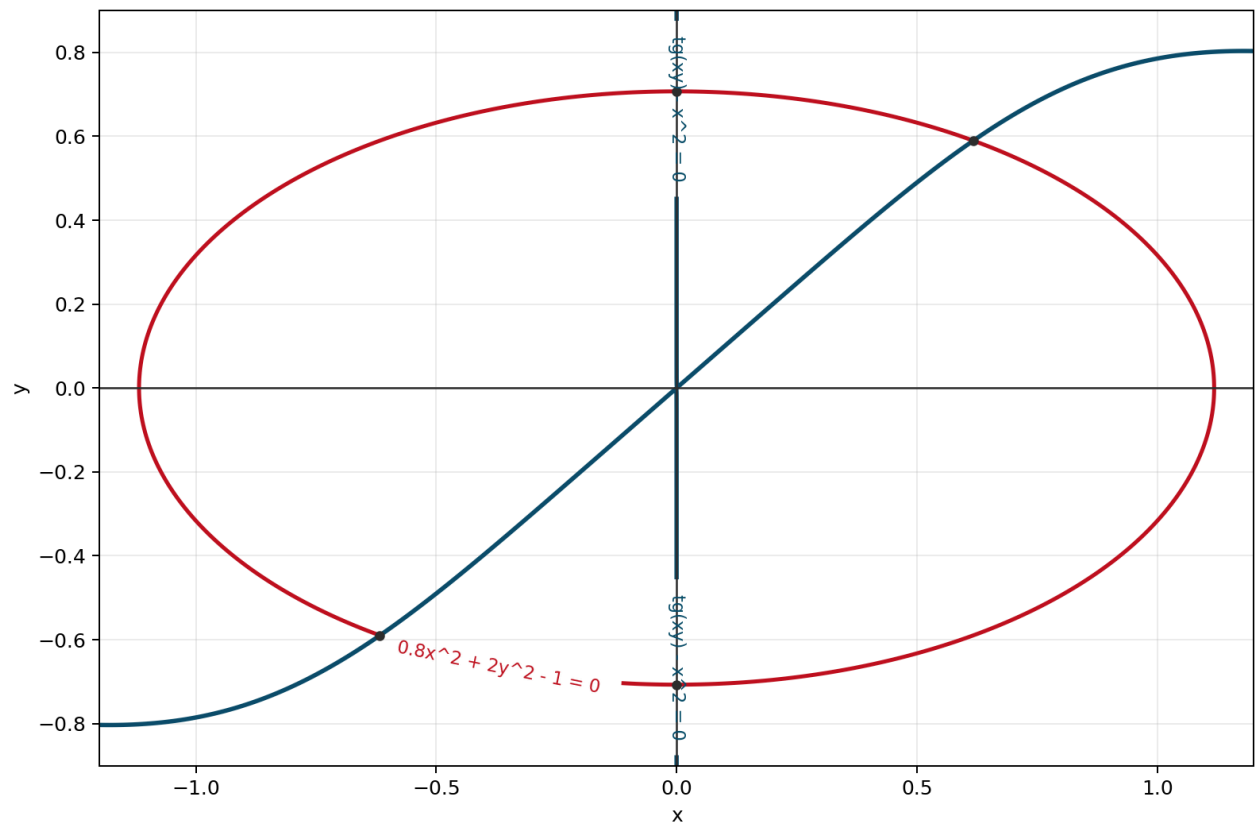
С точностью $\varepsilon = 10^{-2}$ получены корни:

- левый: $x_1 \approx -2.744$,
- центральный: $x_2 \approx 0.499$,
- правый: $x_3 \approx 1.687$.

1.2 Решение системы нелинейных уравнений

$$\begin{cases} \tan(xy) = x^2 \\ 0.8x^2 + 2y^2 = 1 \end{cases} \quad \varepsilon = 10^{-2}$$

1.2.1 Графическое отделение корней системы



По графику видны точки пересечения. Для вычислительного уточнения выберем корень в I четверти с начальным приближением $(x_0, y_0) = (0.700, 0.450)$.

1.2.2 Метод Ньютона для системы

Обозначим:

$$F_1(x, y) = \tan(xy) - x^2,$$

$$F_2(x, y) = 0.8x^2 + 2y^2 - 1.$$

Матрица Якоби:

$$J(x, y) = \begin{pmatrix} y \sec^2(xy) - 2x & x \sec^2(xy) \\ 1.6x & 4y \end{pmatrix}.$$

На каждой итерации решается линейная система $J(x_k, y_k)\delta_k = -F(x_k, y_k)$, после чего

$$x_{k+1} = x_k + \delta x_k,$$

$$y_{k+1} = y_k + \delta y_k.$$

Критерий остановки:

$$\max(|\delta x_k|, |\delta y_k|) < \varepsilon.$$

1.2.3 Итерации Ньютона

k	x_k	y_k	$F_1(x_k, y_k)$	$F_2(x_k, y_k)$	x_{k+1}	y_{k+1}	$\max(\delta x , \delta y)$
1	0.700	0.450	-0.164	-0.203	0.644	0.597	0.147
2	0.644	0.597	-0.010	0.046	0.618	0.590	0.027
3	0.618	0.590	-0.000	0.001	0.617	0.590	0.001

На 3-й итерации выполнено условие точности: $\max(|\delta x|, |\delta y|) = 0.001 < 0.01$.

Итоговое приближение корня системы в I четверти:

$$x \approx 0.617, \quad y \approx 0.590.$$

По симметрии системы также существует корень в III четверти:

$$x \approx -0.617, \quad y \approx -0.590.$$

2 Программная реализация задачи

```
ScalarResult bisection(const ScalarEquation& eq, double a, double b, double eps) {
    ScalarResult res;
    double fa = eq.f(a);
    double fb = eq.f(b);

    if (fa * fb > 0.0) {
        res.message = "Для метода половинного деления требуется смена знака на [a,b].";
        return res;
    }

    double left = a;
    double right = b;
```

```

double mid = 0.5 * (left + right);
double fmid = eq.f(mid);

for (int k = 1; k <= MaxIters; k++) {
    mid = 0.5 * (left + right);
    fmid = eq.f(mid);

    if (fabs(fmid) < eps || fabs(right - left) < eps) {
        res.ok = true;
        res.root = mid;
        res.f_at_root = fmid;
        res.iterations = k;
        res.message = "OK";
        return res;
    }

    if (eq.f(left) * fmid < 0.0) {
        right = mid;
    } else {
        left = mid;
    }
}

res.message = "Достигнуто максимальное число итераций.";
return res;
}

ScalarResult newton_scalar(const ScalarEquation& eq, double a, double b, double eps) {
    ScalarResult res;

    double x = 0.5 * (a + b);
    if (eq.f(a) * eq.d2f(a) > 0.0) {
        x = a;
    } else if (eq.f(b) * eq.d2f(b) > 0.0) {
        x = b;
    }

    for (int k = 1; k <= MaxIters; k++) {
        double dfx = eq.df(x);
        if (fabs(dfx) < EpsMin) {
            res.message = "Производная близка к нулю: метод Ньютона остановлен.";
            return res;
        }

        double x1 = x - eq.f(x) / dfx;
        if (x1 < a || x1 > b) {
            x1 = max(a, min(b, x1));
        }

        if (fabs(x1 - x) < eps || fabs(eq.f(x1)) < eps) {
            res.ok = true;
            res.root = x1;
            res.f_at_root = eq.f(x1);
            res.iterations = k;
            res.message = "OK";
            return res;
        }
        x = x1;
    }
}

```

```

        res.message = "Достигнуто максимальное число итераций.";
        return res;
    }

```

```

ScalarResult simple_iteration_scalar(const ScalarEquation& eq, double a, double b, double
eps) {

```

```

    ScalarResult res;

```

```

    double max_df = max_abs_df(eq.df, a, b);
    if (max_df < EpsMin) {
        res.message = "max|f'(x)| слишком мал, нельзя выбрать lambda.";
        return res;
    }

```

```

    // Находим знак производной на интервале для корректного определения лямбды

```

```

    double df_sign = 0.0;
    for (int i = 0; i <= DefaultSamples; i++) {
        double x = a + (b - a) * i / double(DefaultSamples);
        double df_val = eq.df(x);
        if (fabs(df_val) > EpsMin) {
            df_sign = sign(df_val);
            break;
        }
    }

```

```

    double lambda = -df_sign / max_df;

```

```

    double q = 0.0;
    for (int i = 0; i <= DefaultSamples; i++) {
        double x = a + (b - a) * i / double(DefaultSamples);
        q = max(q, fabs(1.0 + lambda * eq.df(x)));
    }

```

```

    res.lambda = lambda;
    res.q = q;

```

```

    if (q >= 1.0) {
        res.message = "Условие сходимости простой итерации не выполнено: q >= 1.";
        return res;
    }

```

```

    double x = 0.5 * (a + b);
    for (int k = 1; k <= MaxIters; k++) {
        double x1 = x - lambda * eq.f(x);
        if (!isfinite(x1)) {
            res.message = "Итерации разошлись (иррациональное значение).";
            return res;
        }
    }

```

```

    if (fabs(x1 - x) < eps || fabs(eq.f(x1)) < eps) {
        res.ok = true;
        res.root = x1;
        res.f_at_root = eq.f(x1);
        res.iterations = k;
        res.message = "OK";
        return res;
    }
    x = x1;

```

```

    }

    res.message = "Достигнуто максимальное число итераций.";
    return res;
}

SystemResult simple_iteration_system(const SystemDef& s, Point p0, double eps) {
    SystemResult res;

    array<array<double, 2>, 2> j0 = s.j(p0);
    double norm_inf = max(fabs(j0[0][0]) + fabs(j0[0][1]), fabs(j0[1][0]) + fabs(j0[1]
[1]));
    double tau = (norm_inf > EpsMin) ? 1.0 / norm_inf : 1.0;
    double q = 2.0;

    for (int t = 0; t < MaxTauAttempts; t++) {
        double a11 = fabs(1.0 - tau * j0[0][0]) + fabs(-tau * j0[0][1]);
        double a22 = fabs(-tau * j0[1][0]) + fabs(1.0 - tau * j0[1][1]);
        q = max(a11, a22);
        if (q <= 1.0) {
            break;
        }
        tau *= 0.5;
    }

    res.tau = tau;
    res.q = q;

    if (q >= 1.0) {
        res.message =
            "Недостаточное условие сходимости для простой итерации системы не выполнено (q
>= 1).";
        return res;
    }

    Point p = p0;
    for (int k = 1; k <= MaxIters; k++) {
        Point f = s.f(p);
        Point next = {p.x - tau * f.x, p.y - tau * f.y};

        double dx = next.x - p.x;
        double dy = next.y - p.y;

        res.rows.push_back({k, p.x, p.y, dx, dy, f.x, f.y});

        if (!isfinite(next.x) || !isfinite(next.y)) {
            res.message = "Итерации разошлись (иррациональное значение).";
            return res;
        }

        p = next;
        if (max(fabs(dx), fabs(dy)) < eps) {
            res.ok = true;
            res.point = p;
            res.residual = s.f(p);
            res.iterations = k;
            res.message = "OK";
            return res;
        }
    }
}

```

```

    }

    res.message = "Достигнуто максимальное число итераций.";
    return res;
}

```

3 Пример работы программы

===== ЛР2: Нелинейные уравнения/системы =====

1) Решить нелинейное уравнение
 2) Решить систему нелинейных уравнений
 0) Выход
 Ваш выбор: 1

--- Решение нелинейного уравнения ---

1) $f(x) = 3x^3 + 1.7x^2 - 15.42x + 6.89$
 2) $f(x) = \sin(x) - 0.5x$
 3) $f(x) = \exp(-x) - x$
 4) $f(x) = x^3 - x - 1$
 5) $f(x) = \cos(x) - x$

Выберите уравнение [1-5]: 1

Доступные методы:

1) Метод половинного деления
 3) Метод Ньютона
 5) Метод простой итерации

Выберите метод [1/3/5]: 3

Источник входных данных:

1) Клавиатура
 2) Файл

Выберите [1/2]: 1

Введите a: 0

Введите b: 2

Введите epsilon: 0.01

На интервале обнаружено несколько смен знака (возможно несколько корней): 2

Выберите более узкий интервал изоляции одного корня.

===== ЛР2: Нелинейные уравнения/системы =====

1) Решить нелинейное уравнение
 2) Решить систему нелинейных уравнений
 0) Выход
 Ваш выбор: 2

--- Решение системы нелинейных уравнений ---

1) $\{ \tan(xy) = x^2 ; 0.8x^2 + 2y^2 = 1 \}$
 2) $\{ x^2 + y^2 = 1 ; x - y = 0 \}$
 3) $\{ \sin(x + y) - 1.2x = 0 ; x^2 + y - 1 = 0 \}$

Выберите систему [1-3]: 1

Для систем используется только метод 7: простая итерация.

Введите epsilon: 0.01

Введите начальное x_0 : 0.6

Введите начальное y_0 : 0.6

Система: $\{ \tan(xy) = x^2 ; 0.8x^2 + 2y^2 = 1 \}$

Метод: 7

epsilon: 0.0100000000

Начальное приближение: (0.6000000000, 0.6000000000)

Статус: ошибка

Причина: Недостаточное условие сходимости для простой итерации системы не выполнено ($q \geq$

1).

Сохранить результат в файл (y/n): n

Показать ASCII-график системы в терминале (y/n): y

xmin (по умолчанию -2.0):

xmax (по умолчанию 2.0):

ymin (по умолчанию -1.0):

ymax (по умолчанию 1.0):

ASCII-график системы на области x: [-2; 2] y: [-1; 1]

```
.....|.....
.1.....|.....
.....|.....
.....|.....
.....2.....|.....1.....
.....2.....|.....2.....1.....
.....|.....1.....
.....|.....1.....
.....|.....1.....
.....|.....1.....
.....2.....|.....11.....2.....
.....2.....|.....1.....2.....
.....2.....|.....1.....2.....
.....|1111.....
.....2.....1|111.....2.....
.....1|11.....
-----|-----
.....11|1.....
.....2.....111|1.....2.....
.....1111|.....
.....2.....1.....|.....2.....
.....2.....1.....|.....2.....
.....2.....11.....|.....2.....
.....1.....|.....
.....1.....|.....
.....1.....|.....
.....1.....2.....|.....2.....
.....1.....|.....
.....|.....
.....|.....
.....|.....
.....|.....1.....
.....|.....
```

Обозначения: '1' -> F1=0, '2' -> F2=0, 'X' -> пересечение

===== ЛР2: Нелинейные уравнения/системы =====

1) Решить нелинейное уравнение

2) Решить систему нелинейных уравнений

0) Выход

Ваш выбор: 1

--- Решение нелинейного уравнения ---

1) $f(x) = 3x^3 + 1.7x^2 - 15.42x + 6.89$

2) $f(x) = \sin(x) - 0.5x$

3) $f(x) = \exp(-x) - x$

4) $f(x) = x^3 - x - 1$

5) $f(x) = \cos(x) - x$

Выберите уравнение [1-5]: 2

Доступные методы:

1) Метод половинного деления

3) Метод Ньютона

5) Метод простой итерации

Выберите метод [1/3/5]: 5

Источник входных данных:

1) Клавиатура

2) Файл

Выберите [1/2]: 2

Путь к файлу (формат: a b eps в первой строке): input.txt

На интервале обнаружено несколько смен знака (возможно несколько корней): 2

Выберите более узкий интервал изоляции одного корня.

===== ЛР2: Нелинейные уравнения/системы =====

1) Решить нелинейное уравнение

2) Решить систему нелинейных уравнений

0) Выход

Ваш выбор: 1

--- Решение нелинейного уравнения ---

1) $f(x) = 3x^3 + 1.7x^2 - 15.42x + 6.89$

2) $f(x) = \sin(x) - 0.5x$

3) $f(x) = \exp(-x) - x$

4) $f(x) = x^3 - x - 1$

5) $f(x) = \cos(x) - x$

Выберите уравнение [1-5]: 1

Доступные методы:

1) Метод половинного деления

3) Метод Ньютона

5) Метод простой итерации

Выберите метод [1/3/5]: 3

Источник входных данных:

1) Клавиатура

2) Файл

Выберите [1/2]: 2

Путь к файлу (формат: a b eps в первой строке): input.txt

Уравнение: $3x^3 + 1.7x^2 - 15.42x + 6.89$

Метод: 3

Интервал: [-4.0000000000; 0.0000000000]

epsilon: 0.0001000000

Статус: успешно

Корень: -2.7444543890

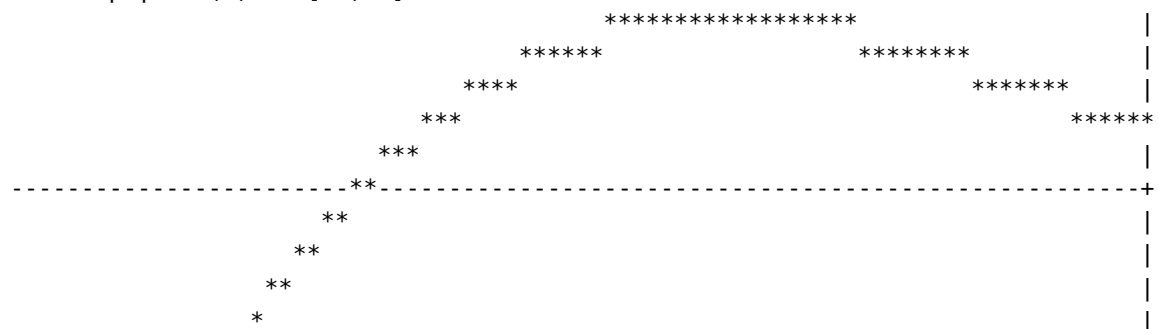
f(корень): -0.0000000002

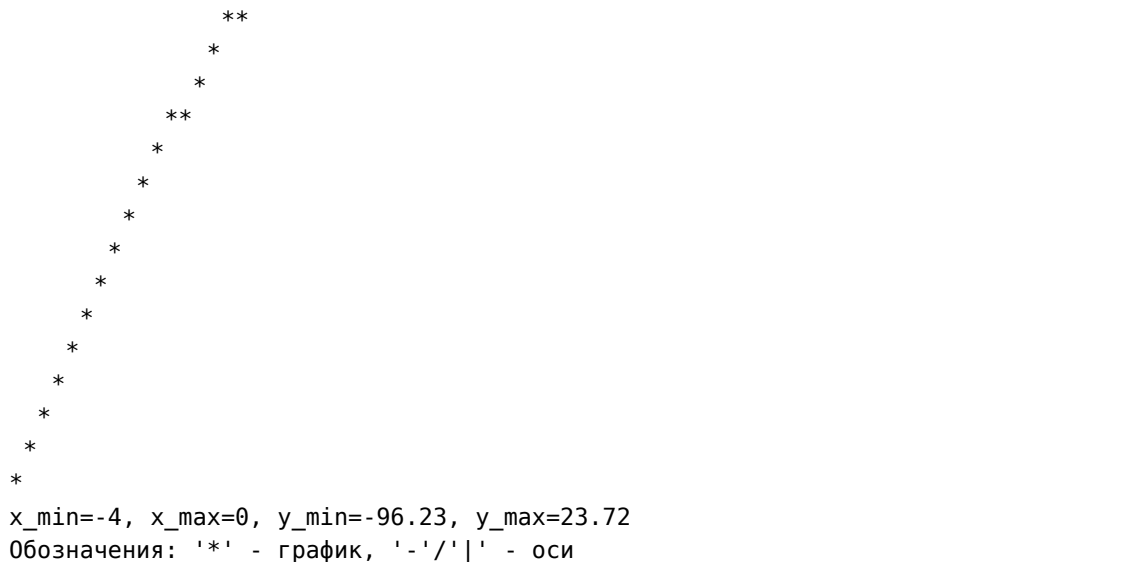
Итераций: 5

Сохранить результат в файл (y/n): n

Показать ASCII-график функции в терминале (y/n): y

ASCII-график f(x) на [-4; 0]





===== ЛР2: Нелинейные уравнения/системы =====

- 1) Решить нелинейное уравнение
- 2) Решить систему нелинейных уравнений
- 0) Выход

Ваш выбор:

4 Выводы

В ходе выполнения лабораторной работы были реализованы и применены численные методы решения нелинейных уравнений и систем.

Для уравнения $f(x) = 3x^3 + 1.7x^2 - 15.42x + 6.89 = 0$ с точностью $\varepsilon = 10^{-2}$ найдены три корня:

- $x_1 \approx -2.744$ — методом хорд за 4 итерации;
- $x_2 \approx 0.499$ — методом секущих за 1 итерацию;
- $x_3 \approx 1.687$ — методом простой итерации за 5 итераций.

Метод Ньютона показал наибольшую скорость сходимости: для корня x_1 потребовалось лишь 3 итерации против 8 у метода бисекции на том же интервале.

Для системы нелинейных уравнений $\begin{cases} \tan(xy)=x^2 \\ 0.8x^2+2y^2=1 \end{cases}$ метод Ньютона сошёлся за 3 итерации, найдя корень $(x, y) \approx (0.617, 0.590)$ в I четверти.

В ходе работы убедились, что:

- метод простой итерации требует выполнения достаточного условия сходимости ($|\varphi'(x)| < 1$), иначе итерации расходятся;
- метод Ньютона обладает квадратичной сходимостью вблизи изолированного корня;
- метод бисекции гарантированно сходится при смене знака функции, но медленнее других методов.